

Programación III

Desarrollo backend con Spring Boot

UNIDAD 1: Programación Funcional

Clase N° 2

Paradigma de programación funcional

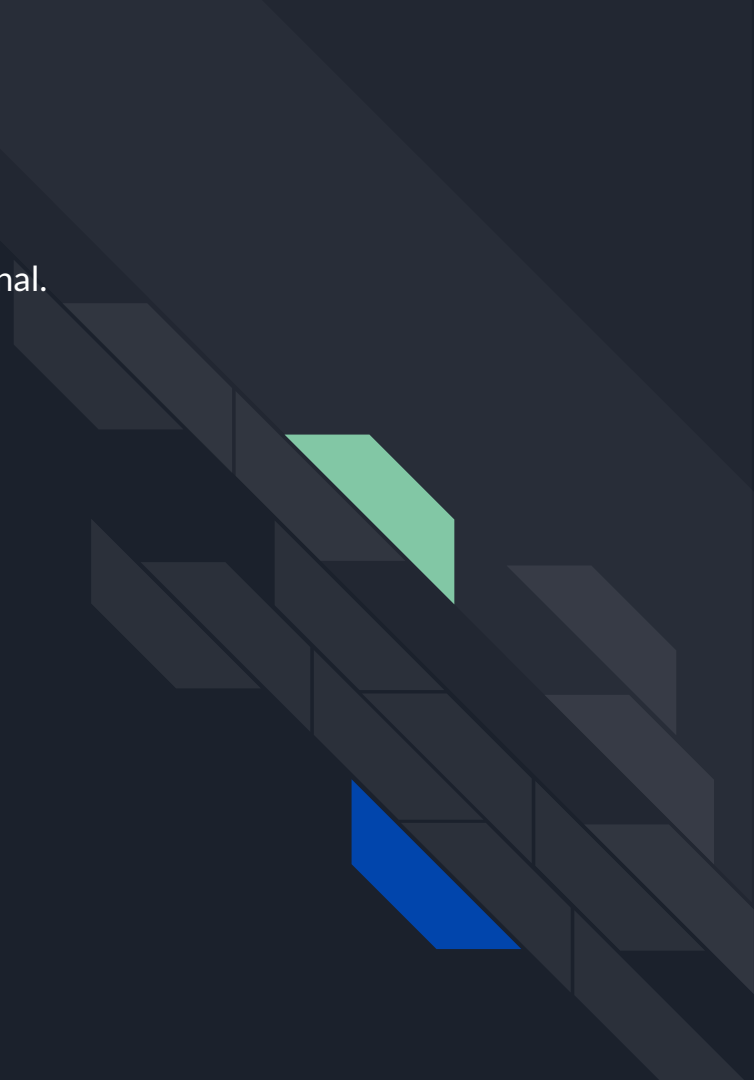
Profesor:

Daniel Díaz



TEMAS A DESARROLLAR

- ¿Qué es la programación funcional?
- Diferencias entre programación imperativa y programación funcional.
- ¿Qué es una expresión lambda y por qué usarla?
- Sintaxis básica (parametros) -> {cuerpo}.
- Interfaces funcionales.
- Uso en expresiones de una sola línea y de varias líneas.
- Introducción a `java.util.function`.
- `Consumer<T>`
- `Supplier<T>`
- `Predicate<T>`
- `Function<T, R>`



Programación Funcional

- Paradigma de programación que trata la computación como la evaluación de funciones matemáticas.
- Evita estados mutables y efectos secundarios.
- En Java, se implementa mediante expresiones lambda, interfaces funcionales y la API de Streams.

Diferencias entre Programación Imperativa y Programación Funcional

Característica	Programación Imperativa	Programación Funcional
Definición	Se basa en instrucciones paso a paso que modifican el estado del programa.	Se basa en la evaluación de funciones matemáticas sin modificar el estado.
Estado Mutable	Modifica variables y estructuras de datos.	Evita cambios en el estado, usa estructuras inmutables.
Efectos Secundarios	Puede tener efectos secundarios (cambiar variables globales, modificar archivos, etc.).	Minimiza efectos secundarios, las funciones son puras .
Enfoque	Se centra en el cómo lograr un resultado (procedimientos y bucles).	Se centra en el qué se quiere lograr (transformaciones de datos).
Ejemplo de Iteración	Usa bucles como <code>for</code> o <code>while</code> .	Usa funciones como <code>map</code> , <code>filter</code> y <code>reduce</code> .

Programación Imperativa

```
public class Imperativa {  
    public static void main(String[] args) {  
        List<Integer> numeros = List.of(1, 2, 3, 4, 5);  
        List<Integer> cuadrados = new ArrayList<>();  
  
        for (int num : numeros) { // Iteración con bucle  
            cuadrados.add(num * num);  
        }  
  
        System.out.println(cuadrados); // Salida: [1, 4, 9, 16, 25]  
    }  
}
```

Programación Funcional

```
public class Funcional {  
    public static void main(String[] args) {  
        List<Integer> numeros = List.of(1, 2, 3, 4, 5);  
  
        List<Integer> cuadrados = numeros.stream()  
                                           .map(num -> num * num) // Transformación funcional  
                                           .collect(Collectors.toList());  
  
        System.out.println(cuadrados); // Salida: [1, 4, 9, 16, 25]  
    }  
}
```

Sintaxis básica de las expresiones lambda

(parametros) -> { cuerpo de la función }

- Paréntesis `()` : Contienen los parámetros de la función.
- Operador `->` : Separa los parámetros del cuerpo de la función.
- Cuerpo `{ }` : Contiene la lógica de la función (puede ser una sola expresión o varias instrucciones).

¿Qué es una interfaz funcional?

Una interfaz funcional en Java es una interfaz que contiene un **único método abstracto**. Se utilizan principalmente para la programación funcional y permiten el uso de expresiones lambda y referencias a métodos.

Características clave:

- ✓ Solo puede tener un método abstracto.
- ✓ Puede contener métodos default y static sin perder su condición de funcional.
- ✓ Se puede marcar con la anotación **@FunctionalInterface** (opcional pero recomendable).


```
@FunctionalInterface
interface Operacion {
    int calcular(int a, int b); // Único método abstracto
}
```

Las interfaces funcionales
pueden tener métodos **default**
y **static** sin afectar su
funcionalidad:



```
@FunctionalInterface
interface Operacion {
    int calcular(int a, int b); // Único método abstracto

    // Método por defecto
    default void mostrarMensaje() {
        System.out.println("Ejecutando operación...");
    }

    // Método estático
    static void mensajeEstatico() {
        System.out.println("Método estático en interfaz.");
    }
}

public class Main {
    public static void main(String[] args) {
        Operacion multiplicacion = (a, b) -> a * b;

        System.out.println(multiplicacion.calcular(4, 2)); // Salida: 8
        multiplicacion.mostrarMensaje(); // Salida: Ejecutando operación...
        Operacion.mensajeEstatico(); // Salida: Método estático en interfaz.
    }
}
```

Expresiones de una línea

```
Operacion suma = (a, b) -> a + b;  
System.out.println(suma.calcular(5, 3)); // Salida: 8
```

- **Operacion** es una interfaz funcional con un único método `calcular(int a, int b)`.
- `a, b` son los parámetros.
- `a + b` es la expresión que se evalúa y devuelve el resultado.

Otro ejemplo...

```
import java.util.function.Function;

Function<String, Integer> longitud = str -> str.length();
System.out.println(longitud.apply("Hola")); // Salida: 4
```

- No se necesitan paréntesis porque hay un solo parámetro (str).
- str.length() es la expresión que devuelve la longitud de la cadena.

Expresiones de más de una línea

```
Operacion multiplicacion = (a, b) -> {  
    int resultado = a * b;  
    System.out.println("Multiplicando " + a + " * " + b);  
    return resultado;  
};  
System.out.println(multiplicacion.calcular(4, 2));
```

- Se usa { } porque hay más de una instrucción.
- Se debe usar return explícitamente en este caso.

Librería `java.util.function`

La librería `java.util.function` es un paquete en Java que contiene una serie de **interfaces funcionales** diseñadas para facilitar la programación funcional mediante expresiones lambda y referencias a métodos.

¿Por qué usar `java.util.function`?

- ✓ Reduce la necesidad de definir interfaces funcionales personalizadas.
- ✓ Mejora la legibilidad y reutilización del código.
- ✓ Compatible con APIs modernas como **Streams** y **Optional**.

Principales interfaces funcionales en java.util.function

- **Consumer<T>** —> Ejecuta una acción sin devolver resultado.
- **Supplier<T>** —> Provee un valor sin recibir argumentos.
- **Predicate<T>** —> Recibe un argumento y devuelve true o false.
- **Function<T, R>** —> Recibe un argumento de tipo T y devuelve un resultado de tipo R.
- **BiFunction<T, U, R>** —> Recibe dos argumentos de tipo T y U, y devuelve un resultado de tipo R.

Consumer <T>

```
import java.util.function.Consumer;

public class Main {
    public static void main(String[] args) {
        Consumer<String> imprimir = mensaje -> System.out.println("Mensaje: " + mensaje);
        imprimir.accept("Hola, Java!"); // Salida: Mensaje: Hola, Java!
    }
}
```

Supplier<T>

```
import java.util.function.Supplier;

public class Main {
    public static void main(String[] args) {
        Supplier<Double> random = () -> Math.random();
        System.out.println(random.get()); // Salida: (Número aleatorio)
    }
}
```


Predicate<T>

```
import java.util.function.Predicate;

public class Main {
    public static void main(String[] args) {
        Predicate<Integer> esPar = num -> num % 2 == 0;
        System.out.println(esPar.test(8)); // Salida: true
    }
}
```

Function<T, R>

```
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        Function<Integer, String> convertir = num -> "Número: " + num;
        System.out.println(convertir.apply(10)); // Salida: Número: 10
    }
}
```

BiFunction<T, U, R>

```
import java.util.function.BiFunction;

public class Main {
    public static void main(String[] args) {
        BiFunction<Integer, Integer, Integer> sumar = (a, b) -> a + b;
        System.out.println(sumar.apply(4, 6)); // Salida: 10
    }
}
```